



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Universidad Nacional de Colombia - sede Bogotá
Facultad de Ingeniería
Departamento de Sistemas e Industrial
Curso: Ingeniería de Software 1 (2016701)

Tutorial de construcción de aplicación

Para este tutorial se requiere la instalación de:

- Lenguaje: Python
- Framework: FastAPI
- ORM: SQLAlchemy
- Base de datos: PostgreSQL
- Servidor ASGI: Uvicorn

Se va a hacer uso de 5 capas principales denominadas

- Controllers
- Services
- Repositories
- Domain
- Database

Teniendo esto en cuenta se va a desarrollar una aplicación básica que muestre el mensaje de Hola Mundo en un servidor, se va a usar únicamente Backend y una base de datos sumado a una prueba desde Postman para la creación y enlistamiento de usuarios.

1. Crear una carpeta en la que se va a alojar la aplicación, dentro de esta se van a crear 5 carpetas con el nombre de las capas anteriormente mencionadas.
2. Abrir una terminal de código, ubicarse dentro de la carpeta de la aplicación y crear un entorno virtual ejecutando los siguientes comandos.

```
python -m venv venv
```

```
venv\Scripts\activate
```

3. Ejecutar el siguiente comando para instalar las dependencias que se van a utilizar.

```
pip install fastapi uvicorn sqlalchemy psycopg2-binary pyyaml
```

4. Abrir la terminal de PostgreSQL y crear la base de datos del sistema usando el siguiente comando.

```
CREATE DATABASE fastapi_db;
```

5. Dentro de la aplicación crear un archivo demoniado **aplicacion.yml** que contenga la siguiente información, la cual va a indicar donde reside la base de datos, la contraseña, el host, usuario, nombre y puerto en el que se va a ejecutar.

```
database:
  host: localhost
  port: 5432
  user: *El usuario de Postgres*
  password: *La contraseña de Postgres*
  name: fastapi_db
```

6. Dentro de la carpeta **database** crear el archivo **connection.py** con el siguiente contenido, el cual establece los parámetros de conexión básicos para usar la base de datos dentro de la aplicación sin usar directamente SQL.

```
from sqlalchemy import create_engine

from sqlalchemy.orm import sessionmaker, declarative_base

import yaml

with open("application.yml", "r") as file:
    config = yaml.safe_load(file)

db = config["database"]

DATABASE_URL =
f"postgresql://{db['user']}:{db['password']}@{db['host']}:{db['port']}/{db['name']}"

engine = create_engine(DATABASE_URL)
```

```
SessionLocal = sessionmaker(bind=engine)
```

```
Base = declarative_base()
```

7. En la carpeta **domain** crear el archivo **usuario.py** con el siguiente contenido que configura la base de datos y establece la forma de la tabla de usuarios y como se van a guardar.

```
from domain.usuario import Usuario
```

```
class UsuarioRepository:
```

```
    def guardar(self, db, nombre):  
        usuario = Usuario(nombre=nombre)
```

```
        db.add(usuario)  
        db.commit()  
        db.refresh(usuario)
```

```
        return usuario
```

```
    def listar(self, db):  
        return db.query(Usuario).all()
```

8. En la carpeta **repositories** crear el archivo **usuario_repository.py** con el siguiente contenido que centraliza la manipulación de datos dentro de la aplicación, con el fin de que si opta por otro ORM distinto al usado solo se tiene editar este archivo para configurar la compatibilidad y no toda la aplicación.

```
from domain.usuario import Usuario
```

```
class UsuarioRepository:
```

```
    def guardar(self, db, nombre):  
        usuario = Usuario(nombre=nombre)
```

```
db.add(usuario)
db.commit()
db.refresh(usuario)
```

```
return usuario
```

```
def listar(self, db):
    return db.query(Usuario).all()
```

9. En la carpeta **services** crear el archivo **usuario_service.py** con el siguiente contenido que configura la lógica de como va a operar la aplicación para guardar los usuarios.

```
from repositories.usuario_repository import UsuarioRepository
```

```
class UsuarioService:
```

```
    def __init__(self):
        self.repository = UsuarioRepository()

    def crear_usuario(self, db, nombre):
        return self.repository.guardar(db, nombre)

    def obtener_usuarios(self, db):
        return self.repository.listar(db)
```

10. En la carpeta **controllers** crear el archivo **usuario_controller.py** con el siguiente contenido que establece el método get al APIRouter para mostrar el mensaje Hola mundo y el post para crear el usuario retornando luego el nombre e id, generando una lista finalmente.

```
from fastapi import APIRouter
from database.connection import SessionLocal
from services.usuario_service import UsuarioService
```

```
router = APIRouter()
```

```
service = UsuarioService()
```

```
@router.get("/hola")
def hola_mundo():
    return {"mensaje": "Hola Mundo"}
```

```
@router.post("/usuarios/{nombre}")
def crear_usuario(nombre: str):
```

```
    db = SessionLocal()
```

```
    usuario = service.crear_usuario(db, nombre)
```

```
    db.close()
```

```
    return {
        "id": usuario.id,
        "nombre": usuario.nombre
    }
```

```
@router.get("/usuarios")
def listar_usuarios():
```

```
    db = SessionLocal()
```

```
    usuarios = service.obtener_usuarios(db)
```

```
    db.close()
```

```
return usuarios
```

11. Dentro de la carpeta madre de la aplicación crear el archivo principal denominado **main.py** que publica el servidor y conecta la base de datos para ejecutar la aplicación.

```
from fastapi import FastAPI
```

```
from controllers.usuario_controller import router
```

```
from database.connection import engine, Base
```

```
app = FastAPI()
```

```
Base.metadata.create_all(bind=engine)
```

```
app.include_router(router)
```

12. En la terminal donde se creo el entorno virtual y se realizó la instalación de las dependencias ejecutar el siguiente comando para crear el servidor y ejecutar la aplicación.

```
uvicorn *El nombre de la aplicación*.main:app --reload
```

13. En el navegador abrir el siguiente enlace en el cual se debe observar el mensaje Hola mundo

<http://127.0.0.1:8000/hola>

14. Abrir el siguiente enlace para ejecutar la prueba POST para agregar un usuario, FastAPI integra una herramienta denominada Swagger UI para ejecutar este tipo de pruebas viendo los endpoints en la documentación interactiva.

<http://127.0.0.1:8000/docs>

Los comandos de la terminal son para el sistema operativo de Windows.

Adicionalmente se puede crear un archivo denominado **requirements.txt** que contenga las dependencias usadas, esto permite que si se trabaja la aplicación en conjunto el grupo de trabajo cree un entorno virtual y ejecute un simple comando para descargar las dependencias y que la aplicación funcione correctamente.

```
pip install -r requirements.txt
```